

Dokumentowe bazy danych – MongoDB

Ćwiczenie 2

Imiona i nazwiska autorów: Krystian Augustyn, Jakub Węgrzyniak

Odtwórz z backupu bazę **north0**

- najprostsza wersja

```
mongorestore dump
```

- to polecenie odtworzy wszystkie bazy danych znajdujące się we wskazanym folderze (w tym przypadku **dump**)
 - najłatwiej wgrać tam folder zawierający pliki z backupem i wykonać proste polecenie `mongorestore`
- dokumentacja:
 - <https://www.mongodb.com/docs/database-tools/mongorestore/>

Wybierz bazę **north0**

Baza **north0** jest kopią relacyjnej bazy danych **Northwind**

- poszczególne kolekcje odpowiadają tabelom w oryginalnej bazie **Northwind**

Zadanie 0

zapoznaj się ze strukturą dokumentów w bazie **North0**

```
db.customers.find()  
db.orders.find();  
db.orderdetails.find();
```

Zadanie 1 – operacje wyszukiwania danych, przetwarzanie dokumentów

a)

stwórz kolekcję **OrdersInfo** zawierającą następujące dane o zamówieniach

- kolekcję **OrdersInfo** należy stworzyć przekształcając dokumenty w oryginalnych kolekcjach **customers**, **orders**, **orderdetails**, **employees**, **shippers**, **products**, **categories**, **suppliers** do kolekcji w której pojedynczy dokument opisuje jedno zamówienie

spodziewany wynik:

```
[
  {
    "_id": ...

    OrderID": ... numer zamówienia

    "Customer": { ... podstawowe informacje o kliencie składającym
      "CustomerID": ... identyfikator klienta
      "CompanyName": ... nazwa klienta
      "City": ... miasto
      "Country": ... kraj
    },

    "Employee": { ... podstawowe informacje o pracowniku obsługującym
zamówienie
      "EmployeeID": ... idntyfikator pracownika
      "FirstName": ... imie
      "LastName": ... nazwisko
      "Title": ... stanowisko
    },

    "Dates": {
      "OrderDate": ... data złożenia zamówienia
      "RequiredDate": data wymaganej realizacji
    }

    "Orderdetails": [ ... pozycje/szczegóły zamówienia – tablica takich
pozycji
      {
        "UnitPrice": ... cena
        "Quantity": ... liczba sprzedanych jednostek towaru
        "Discount": ... zniżka
        "Value": ... wartość pozycji zamówienia
        "product": { ... podstawowe informacje o produkcie
          "ProductID": ... identyfikator produktu
          "ProductName": ... nazwa produktu
          "QuantityPerUnit": ... opis/opakowanie
          "CategoryID": ... identyfikator kategorii do której należy
produkt
          "CategoryName" ... nazwę tej kategorii
        },
      },
      ...
    ],
  },
]
```

```

"Freight": ... opłata za przesyłkę
"OrderTotal" ... sumaryczna wartosc sprzedanych produktów

"Shipment" : { ... informacja o wysyłce
  "Shipper": { ... podstawowe inf o przewoźniku
    "ShipperID":
    "CompanyName":
  }
  ... inf o odbiorcy przesyłki
  "ShipName": ...
  "ShipAddress": ...
  "ShipCity": ...
  "ShipCountry": ...
}
}
]

```

b)

stwórz kolekcję **CustomerInfo** zawierającą następujące dane każdym klientie

- pojedynczy dokument opisuje jednego klienta

spodziewany wynik:

```

[
  {
    "_id": ...

    "CustomerID": ... identyfikator klienta
    "CompanyName": ... nazwa klienta
    "City": ... miasto
    "Country": ... kraj

    "Orders": [ ... tablica zamówień klienta o strukturze takiej jak w
punkcie a)
                (oczywiście bez informacji o kliencie)

  ]
]

```

c)

Napisz polecenie/zapytanie: Dla każdego klienta pokaż wartość zakupionych przez niego produktów z kategorii 'Confections' w 1997r

- Spróbuj napisać to zapytanie wykorzystując
 - oryginalne kolekcje (`customers`, `orders`, `orderdetails`, `products`, `categories`)
 - kolekcję `OrderInfo`
 - kolekcję `CustomerInfo`
- porównaj zapytania/polecenia/wyniki

```
[
  {
    "_id":

    "CustomerID": ... identyfikator klienta
    "CompanyName": ... nazwa klienta
    "ConfectionsSale97": ... wartość zakupionych przez niego produktów
                        z kategorii 'Confections' w 1997r

  }
]
```

d)

Napisz polecenie/zapytanie: Dla każdego klienta poaje wartość sprzedaży z podziałem na lata i miesiące
Spróbuj napisać to zapytanie wykorzystując - oryginalne kolekcje (`customers`, `orders`, `orderdetails`, `products`, `categories`) - kolekcję `OrderInfo` - kolekcję `CustomerInfo`

- porównaj zapytania/polecenia/wyniki

```
[
  {
    "_id":

    "CustomerID": ... identyfikator klienta
    "CompanyName": ... nazwa klienta

    "Sale": [ ... tablica zawierająca inf o sprzedaży
      {
        "Year": ....
        "Month": ....
        "Total": ...
      }
      ...
    ]
  }
]
```

e)

Założmy że pojawia się nowe zamówienie dla klienta 'ALFKI', zawierające dwa produkty 'Chai' oraz "Ikura"

- pozostałe pola w zamówieniu (ceny, liczby sztuk prod, inf o przewoźniku itp. możesz uzupełnić wg własnego uznania) Napisz polecenie które dodaje takie zamówienie do bazy
- aktualizując oryginalne kolekcje `orders`, `orderdetails`
- aktualizując kolekcję `OrderInfo`
- aktualizując kolekcję `CustomerInfo`

Napisz polecenie

- aktualizując oryginalną kolekcję `orderdetails``
- aktualizując kolekcję `OrderInfo`
- aktualizując kolekcję `CustomerInfo`

f)

Napisz polecenie które modyfikuje zamówienie dodane w pkt e) zwiększając zniżkę o 5% (dla każdej pozycji tego zamówienia)

Napisz polecenie

- aktualizując oryginalną kolekcję `orderdetails`
- aktualizując kolekcję `OrderInfo`
- aktualizując kolekcję `CustomerInfo`

UWAGA: W raporcie należy zamieścić kod poleceń oraz uzyskany rezultat, np wynik polecenia

`db.kolekccka.find().limit(2)` lub jego fragment

Zadanie 1 - rozwiązanie

Wyniki:

przykłady, kod, zrzuty ekranów, komentarz ...

a)

```
db.orders.aggregate([
  // dane klienta
  {
    $lookup: {
      from: "customers",
      localField: "CustomerID",
      foreignField: "CustomerID",
      as: "CustomerData"
    }
  },
  { $unwind: "$CustomerData" },
```

```
// dane pracownika
{
  $lookup: {
    from: "employees",
    localField: "EmployeeID",
    foreignField: "EmployeeID",
    as: "EmployeeData"
  }
},
{ $unwind: "$EmployeeData" },

// Dołączamy przewoźnika
{
  $lookup: {
    from: "shippers",
    localField: "ShipVia",
    foreignField: "ShipperID",
    as: "ShipperData"
  }
},
{ $unwind: { path: "$ShipperData", preserveNullAndEmptyArrays: true } },

// Dołączamy szczegóły zamówienia (orderdetails)
{
  $lookup: {
    from: "orderdetails",
    localField: "OrderID",
    foreignField: "OrderID",
    as: "OrderDetailsArray"
  }
},
{ $unwind: "$OrderDetailsArray" },

// Dołączamy produkt do zamówienia
{
  $lookup: {
    from: "products",
    localField: "OrderDetailsArray.ProductID",
    foreignField: "ProductID",
    as: "ProductData"
  }
},
{ $unwind: "$ProductData" },

// Dołączamy kategorię do produktu
{
  $lookup: {
    from: "categories",
    localField: "ProductData.CategoryID",
    foreignField: "CategoryID",
    as: "CategoryData"
  }
},
},
```

```
{ $unwind: "$CategoryData" },

// Obliczamy wartość (Value = UnitPrice * Quantity * (1 - Discount))i
budujemy obiekt "product"
{
  $addFields: {
    "OrderDetailsArray.Value": {
      $multiply: [
        "$OrderDetailsArray.UnitPrice",
        "$OrderDetailsArray.Quantity",
        { $subtract: [1, "$OrderDetailsArray.Discount"] }
      ]
    },
    "OrderDetailsArray.product": {
      ProductID: "$ProductData.ProductID",
      ProductName: "$ProductData.ProductName",
      QuantityPerUnit: "$ProductData.QuantityPerUnit",
      CategoryID: "$CategoryData.CategoryID",
      CategoryName: "$CategoryData.CategoryName"
    }
  }
},

// Zwijamy wszystko z powrotem do pojedynczego zamówienia i układamy
strukturę
{
  $group: {
    _id: "$_id",
    OrderID: { $first: "$OrderID" },
    Customer: {
      $first: {
        CustomerID: "$CustomerData.CustomerID",
        CompanyName: "$CustomerData.CompanyName",
        City: "$CustomerData.City",
        Country: "$CustomerData.Country"
      }
    },
    Employee: {
      $first: {
        EmployeeID: "$EmployeeData.EmployeeID",
        FirstName: "$EmployeeData.FirstName",
        LastName: "$EmployeeData.LastName",
        Title: "$EmployeeData.Title"
      }
    },
    Dates: {
      $first: {
        OrderDate: "$OrderDate",
        RequiredDate: "$RequiredDate"
      }
    },
    Orderdetails: {
      $push: {
        UnitPrice: "$OrderDetailsArray.UnitPrice",
```

```

        Quantity: "$OrderDetailsArray.Quantity",
        Discount: "$OrderDetailsArray.Discount",
        Value: "$OrderDetailsArray.Value",
        product: "$OrderDetailsArray.product"
    }
},
Freight: { $first: "$Freight" },
OrderTotal: { $sum: "$OrderDetailsArray.Value" },
Shipment: {
    $first: {
        Shipper: {
            ShipperID: "$ShipperData.ShipperID",
            CompanyName: "$ShipperData.CompanyName"
        },
        ShipName: "$ShipName",
        ShipAddress: "$ShipAddress",
        ShipCity: "$ShipCity",
        ShipCountry: "$ShipCountry"
    }
}
},
},
// ostateczny wygląd zgodny z wynikiem
{
    $project: {
        _id: 1,
        OrderID: 1,
        Customer: 1,
        Employee: 1,
        Dates: 1,
        Orderdetails: 1,
        Freight: 1,
        "Order Total": "$OrderTotal",
        Shipment: 1
    }
},
// Zapisujemy wynik agregacji do nowej kolekcji OrdersInfo
{ $out: "OrdersInfo" }
])

```

Polecenie weryfikujące poprawność

```
db.OrdersInfo.find().limit(1).pretty()
```

Wyniki:


```
{
  _id: ObjectId('63a060b9bb3b972d6f4e217d'),
  OrderID: 10687,
  Customer: {
    CustomerID: 'HUNGO',
    CompanyName: 'Hungry Owl All-Night Grocers',
    City: 'Cork',
    Country: 'Ireland'
  },
  Employee: {
    EmployeeID: 9,
    FirstName: 'Anne',
    LastName: 'Dodsworth',
    Title: 'Sales Representative'
  },
  Dates: {
    OrderDate: 1997-09-30T00:00:00.000Z,
    RequiredDate: 1997-10-28T00:00:00.000Z
  },
  Orderdetails: [
    {
      UnitPrice: 97,
      Quantity: 50,
      Discount: 0.25,
      Value: 3637.5,
      product: {
        ProductID: 9,
        ProductName: 'Mishi Kobe Niku',
        QuantityPerUnit: '18 - 500 g pkgs.',
        CategoryID: 6,
        CategoryName: 'Meat/Poultry'
      }
    },
    {
      UnitPrice: 123.79,
      Quantity: 10,
      Discount: 0,
      Value: 1237.9,
      product: {
        ProductID: 29,
        ProductName: 'Thüringer Rostbratwurst',
        QuantityPerUnit: '50 bags x 30 sausgs.',
        CategoryID: 6,
        CategoryName: 'Meat/Poultry'
      }
    },
    {
      UnitPrice: 19,
      Quantity: 6,
      Discount: 0.25,
      Value: 85.5,
      product: {
```

```

        ProductID: 36,
        ProductName: 'Inlagd Sill',
        QuantityPerUnit: '24 - 250 g jars',
        CategoryID: 8,
        CategoryName: 'Seafood'
    }
}
],
Freight: 296.43,
Shipment: {
    Shipper: {
        ShipperID: 2,
        CompanyName: 'United Package'
    },
    ShipName: 'Hungry Owl All-Night Grocers',
    ShipAddress: '8 Johnstown Road',
    ShipCity: 'Cork',
    ShipCountry: 'Ireland'
},
'Order Total': 4960.9
}

```

b)

```

db.customers.aggregate([
    // Dołączamy całą listę zamówień z wcześniej stworzonej kolekcji
    OrdersInfo
    {
        $lookup: {
            from: "OrdersInfo",
            localField: "CustomerID",
            foreignField: "Customer.CustomerID",
            as: "Orders"
        }
    },

    // ostateczny wygląd dokumentu i usuwamy zduplikowane dane klienta z
    zamówień
    {
        $project: {
            _id: 1,
            CustomerID: 1,
            CompanyName: 1,
            City: 1,
            Country: 1,
            "Orders._id": 1,
            "Orders.OrderID": 1,
            "Orders.Employee": 1,
            "Orders.Dates": 1,
            "Orders.Orderdetails": 1,
            "Orders.Freight": 1,
            "Orders.Order Total": 1,

```

```

        "Orders.Shipment": 1
    }
},

// Zapisujemy wynik do kolekcji CustomerInfo
{ $out: "CustomerInfo" }
])

```

Polecenie weryfikujące poprawność

```
db.CustomerInfo.find().limit(1).pretty()
```

Wyniki:

```

{
  _id: ObjectId('63a05cdfbb3b972d6f4e097b'),
  CustomerID: 'ALFKI',
  CompanyName: 'Alfreds Futterkiste',
  City: 'Berlin',
  Country: 'Germany',
  Orders: [ // tablica zamówień klienta o strukturze takiej jak w punkcie
a) (oczywiście bez informacji o kliencie) (dodałem tutaj jedno zamówienie
dla czytelności)
    {
      _id: ObjectId('63a060b9bb3b972d6f4e218c'),
      OrderID: 10702,
      Employee: {
        EmployeeID: 4,
        FirstName: 'Margaret',
        LastName: 'Peacock',
        Title: 'Sales Representative'
      },
      Dates: {
        OrderDate: 1997-10-13T00:00:00.000Z,
        RequiredDate: 1997-11-24T00:00:00.000Z
      },
      Orderdetails: [
        {
          UnitPrice: 10,
          Quantity: 6,
          Discount: 0,
          Value: 60,
          product: {
            ProductID: 3,
            ProductName: 'Aniseed Syrup',
            QuantityPerUnit: '12 - 550 ml bottles',
            CategoryID: 2,
            CategoryName: 'Condiments'
          }
        }
      ]
    }
  ]
}

```

```

    },
    {
      UnitPrice: 18,
      Quantity: 15,
      Discount: 0,
      Value: 270,
      product: {
        ProductID: 76,
        ProductName: 'Lakkalikööri',
        QuantityPerUnit: '500 ml',
        CategoryID: 1,
        CategoryName: 'Beverages'
      }
    }
  ],
  Freight: 23.94,
  Shipment: {
    Shipper: {
      ShipperID: 1,
      CompanyName: 'Speedy Express'
    },
    ShipName: "Alfred's Futterkiste",
    ShipAddress: 'Obere Str. 57',
    ShipCity: 'Berlin',
    ShipCountry: 'Germany'
  },
  'Order Total': 330
},
]
}

```

c)

Użycie oryginalnych kolekcji wymagałoby bardzo złożonego zapytania z kilkoma złączeniami (\$lookup), co jest powolne. Z kolekcją OrdersInfo byłoby łatwiej, ale nadal wymagałoby dodatkowego grupowania. Wybrane przeze mnie rozwiązanie na kolekcji CustomerInfo jest najprostsze i najszybsze – wystarczy tylko rozwinąć zagnieżdżone tablice, przefiltrować dane i zsumować wyniki.

```

db.CustomerInfo.aggregate([
  // Rozwijamy tablicę zamówień klienta
  { $unwind: "$Orders" },

  // Filtrujemy tylko zamówienia z 1997 roku
  {
    $match: {
      "Orders.Dates.OrderDate": {
        $gte: ISODate("1997-01-01T00:00:00Z"),
        $lt: ISODate("1998-01-01T00:00:00Z")
      }
    }
  }
],

```

```
// Rozwijamy pozycje w zamówieniu, aby sprawdzić kategorie produktów
{ $unwind: "$Orders.Orderdetails" },

// Filtrujemy tylko produkty z kategorii Confections
{
  $match: {
    "Orders.Orderdetails.product.CategoryName": "Confections"
  }
},

// Grupujemy i sumujemy wartość
{
  $group: {
    _id: "$_id",
    CustomerID: { $first: "$CustomerID" },
    CompanyName: { $first: "$CompanyName" },
    ConfectionsSale97: { $sum: "$Orders.Orderdetails.Value" }
  }
},

// Formatujemy wynik
{
  $project: {
    _id: 1,
    CustomerID: 1,
    CompanyName: 1,
    ConfectionsSale97: 1
  }
}
])
```

Przykładowe wyniki:

```
{
  _id: ObjectId('63a05cdfbb3b972d6f4e097e'),
  CustomerID: 'AROUT',
  CompanyName: 'Around the Horn',
  ConfectionsSale97: 375.19999977201223
}
{
  _id: ObjectId('63a05cdfbb3b972d6f4e09c1'),
  CustomerID: 'SAVEA',
  CompanyName: 'Save-a-lot Markets',
  ConfectionsSale97: 6351.084993118047
}
```

d)

Podobnie jak wcześniej, najwydajniejsza jest gotowa kolekcja CustomerInfo, ponieważ pozwala nam uniknąć czasochłonnego łączenia tabel. Żeby uzyskać podział na lata i miesiące, po prostu wyciągamy daty z zamówień i wykonujemy dwa szybkie grupowania (\$group): najpierw sumując sprzedaż dla danego miesiąca, a potem zwijając to wszystko do jednej tablicy dla każdego klienta.

```
db.CustomerInfo.aggregate([
  // Rozwijamy zamówienia
  { $unwind: "$Orders" },

  // Rozwijamy szczegóły zamówień, aby uzyskać wartość
  { $unwind: "$Orders.Orderdetails" },

  // Sumujemy sprzedaż dla każdej kombinacji (Klient + Rok + Miesiąc)
  {
    $group: {
      _id: {
        CustomerID: "$CustomerID",
        CompanyName: "$CompanyName",
        Year: { $year: { $toDate: "$Orders.Dates.OrderDate" } },
        Month: { $month: { $toDate: "$Orders.Dates.OrderDate" } }
      },
      MonthlyTotal: { $sum: "$Orders.Orderdetails.Value" }
    }
  },

  // Zwijamy wyliczone miesiące z powrotem do tablicy 'Sale' dla klienta
  {
    $group: {
      _id: "$_id.CustomerID",
      CustomerID: { $first: "$_id.CustomerID" },
      CompanyName: { $first: "$_id.CompanyName" },
      Sale: {
        $push: {
          Year: "$_id.Year",
          Month: "$_id.Month",
          Total: "$MonthlyTotal"
        }
      }
    }
  },

  // Formatowanie wyglądu
  {
    $project: {
      _id: 0,
      CustomerID: 1,
      CompanyName: 1,
      Sale: 1
    }
  }
])
```

Przykładowe wyniki:

```
{
  CustomerID: 'LILAS',
  CompanyName: 'LILA-Supermercado',
  Sale: [
    {
      Year: 1998,
      Month: 4,
      Total: 1885
    },
    {
      Year: 1997,
      Month: 4,
      Total: 1412
    },
    {
      Year: 1996,
      Month: 10,
      Total: 1648.999988436699
    },
    {
      Year: 1998,
      Month: 5,
      Total: 673.9199996200203
    },
    {
      Year: 1997,
      Month: 12,
      Total: 720
    },
    {
      Year: 1996,
      Month: 9,
      Total: 1050.6
    },
    {
      Year: 1996,
      Month: 12,
      Total: 112
    },
    {
      Year: 1996,
      Month: 11,
      Total: 1167.6799971342086
    },
    {
      Year: 1997,
      Month: 5,
      Total: 1504.4999894499779
    },
    {
      Year: 1996,
```

```

    Month: 8,
    Total: 1414.8
  },
  {
    Year: 1998,
    Month: 1,
    Total: 2825.999995805323
  },
  {
    Year: 1998,
    Month: 2,
    Total: 122.39999914169312
  },
  {
    Year: 1997,
    Month: 2,
    Total: 1538.7
  }
]
}
```

e)

- Aktualizacja oryginalnych kolekcji (orders, orderdetails):

```

// Dodanie nagłówka zamówienia
db.orders.insertOne({
  OrderID: 99999,
  CustomerID: "ALFKI",
  EmployeeID: 1,
  OrderDate: new ISODate("2026-05-04T00:00:00Z"),
  ShipVia: 1,
  Freight: 15.50,
  ShipName: "Alfred's Futterkiste",
  ShipAddress: "Obere Str. 57",
  ShipCity: "Berlin",
  ShipCountry: "Germany"
});

// Dodanie dwóch produktów do zamówienia
db.orderdetails.insertMany([
  { OrderID: 99999, ProductID: 1, UnitPrice: 18.00, Quantity: 10, Discount:
0 },
  { OrderID: 99999, ProductID: 10, UnitPrice: 31.00, Quantity: 5, Discount:
0 }
]);
```

- Aktualizacja kolekcji OrdersInfo:


```
// Wstawienie całego dokumentu
db.OrdersInfo.insertOne({
  OrderID: 99999,
  Customer: {
    CustomerID: "ALFKI",
    CompanyName: "Alfreds Futterkiste",
    City: "Berlin",
    Country: "Germany"
  },
  Employee: {
    EmployeeID: 1,
    FirstName: "Nancy",
    LastName: "Davolio",
    Title: "Sales Representative"
  },
  Dates: {
    OrderDate: new ISODate("2026-05-04T00:00:00Z"),
    RequiredDate: new ISODate("2026-05-18T00:00:00Z")
  },
  Orderdetails: [
    {
      UnitPrice: 18.00,
      Quantity: 10,
      Discount: 0,
      Value: 180.00,
      product: {
        ProductID: 1,
        ProductName: "Chai",
        QuantityPerUnit: "10 boxes x 20 bags",
        CategoryID: 1,
        CategoryName: "Beverages"
      }
    },
    {
      UnitPrice: 31.00,
      Quantity: 5,
      Discount: 0,
      Value: 155.00,
      product: {
        ProductID: 10,
        ProductName: "Ikura",
        QuantityPerUnit: "12 - 200 ml jars",
        CategoryID: 8,
        CategoryName: "Seafood"
      }
    }
  ],
  Freight: 15.50,
  "Order Total": 335.00,
  Shipment: {
    Shipper: {
      ShipperID: 1,
```

```

        CompanyName: "Speedy Express"
    },
    ShipName: "Alfred's Futterkiste",
    ShipAddress: "Obere Str. 57",
    ShipCity: "Berlin",
    ShipCountry: "Germany"
}
});

```

- Aktualizacja kolekcji CustomerInfo:

```

// Wypchnięcie nowego zamówienia do istniejącej tablicy Orders klienta ALFKI
db.CustomerInfo.updateOne(
  { CustomerID: "ALFKI" },
  {
    $push: {
      Orders: {
        OrderID: 99999,
        Employee: {
          EmployeeID: 1,
          FirstName: "Nancy",
          LastName: "Davolio",
          Title: "Sales Representative"
        },
        Dates: {
          OrderDate: new ISODate("2026-05-04T00:00:00Z"),
          RequiredDate: new ISODate("2026-05-18T00:00:00Z")
        },
        Orderdetails: [
          {
            UnitPrice: 18.00,
            Quantity: 10,
            Discount: 0,
            Value: 180.00,
            product: {
              ProductID: 1,
              ProductName: "Chai",
              QuantityPerUnit: "10 boxes x 20 bags",
              CategoryID: 1,
              CategoryName: "Beverages"
            }
          },
          {
            UnitPrice: 31.00,
            Quantity: 5,
            Discount: 0,
            Value: 155.00,
            product: {
              ProductID: 10,
              ProductName: "Ikura",
              QuantityPerUnit: "12 - 200 ml jars",

```

```

        CategoryID: 8,
        CategoryName: "Seafood"
    }
}
],
Freight: 15.50,
"Order Total": 335.00,
Shipment: {
    Shipper: {
        ShipperID: 1,
        CompanyName: "Speedy Express"
    },
    ShipName: "Alfred's Futterkiste",
    ShipAddress: "Obere Str. 57",
    ShipCity: "Berlin",
    ShipCountry: "Germany"
}
}
}
);

```

Polecenie weryfikujące poprawność

```

db.CustomerInfo.find(
  { CustomerID: "ALFKI" },
  { Orders: { $slice: -1 } } // ostatni element
).pretty()

```

Wyniki:

```

{
  _id: ObjectId('63a05cdfbb3b972d6f4e097b'),
  CustomerID: 'ALFKI',
  CompanyName: 'Alfreds Futterkiste',
  City: 'Berlin',
  Country: 'Germany',
  Orders: [
    {
      OrderID: 99999,
      Employee: {
        EmployeeID: 1,
        FirstName: 'Nancy',
        LastName: 'Davolio',
        Title: 'Sales Representative'
      },
      Dates: {
        OrderDate: 2026-05-04T00:00:00.000Z,
        RequiredDate: 2026-05-18T00:00:00.000Z
      }
    }
  ]
}

```

```

    },
    Orderdetails: [
      {
        UnitPrice: 18,
        Quantity: 10,
        Discount: 0,
        Value: 180,
        product: {
          ProductID: 1,
          ProductName: 'Chai',
          QuantityPerUnit: '10 boxes x 20 bags',
          CategoryID: 1,
          CategoryName: 'Beverages'
        }
      },
      {
        UnitPrice: 31,
        Quantity: 5,
        Discount: 0,
        Value: 155,
        product: {
          ProductID: 10,
          ProductName: 'Ikura',
          QuantityPerUnit: '12 - 200 ml jars',
          CategoryID: 8,
          CategoryName: 'Seafood'
        }
      }
    ],
    Freight: 15.5,
    'Order Total': 335,
    Shipment: {
      Shipper: {
        ShipperID: 1,
        CompanyName: 'Speedy Express'
      },
      ShipName: "Alfred's Futterkiste",
      ShipAddress: 'Obere Str. 57',
      ShipCity: 'Berlin',
      ShipCountry: 'Germany'
    }
  }
]
}

```

f)

- Aktualizacja w oryginalnej kolekcji **orderdetails**:

```

db.orderdetails.updateMany(
  { OrderID: 99999 },

```

```
{ $inc: { Discount: 0.05 } }  
);
```

- Aktualizacja w kolekcji **OrdersInfo**:

```
db.OrdersInfo.updateOne(  
  { OrderID: 99999 },  
  { $inc: { "Orderdetails.$[].Discount": 0.05 } }  
  // $[] sprawia, że zmiana dotyczy każdego elementu w tablicy Orderdetails  
);
```

- Aktualizacja w kolekcji **CustomerInfo**:

```
db.CustomerInfo.updateOne(  
  { CustomerID: "ALFKI" },  
  { $inc: { "Orders.$[order].Orderdetails.$[].Discount": 0.05 } },  
  { arrayFilters: [ { "order.OrderID": 99999 } ] }  
  // arrayFilters wybiera zamówienie o ID 99999 z tablicy Orders, a  
  następnie modyfikujemy wszystkie ($[]) pozycje w jego Orderdetails  
);
```

Polecenie weryfikujące poprawność

```
db.CustomerInfo.find(  
  { CustomerID: "ALFKI" },  
  { Orders: { $slice: -1 } } //ostatni element  
)<pre>
.pretty()
```

Wyniki:

```
{  
  _id: ObjectId('63a05cdfbb3b972d6f4e097b'),  
  CustomerID: 'ALFKI',  
  CompanyName: 'Alfreds Futterkiste',  
  City: 'Berlin',  
  Country: 'Germany',  
  Orders: [  
    {  
      OrderID: 99999,  
      Employee: {  
        EmployeeID: 1,  
        FirstName: 'Nancy',  
        LastName: 'Davolio',  
        Title: 'Sales Representative'  
      },  
    },  
  ],  
}
```

```
Dates: {
  OrderDate: 2026-05-04T00:00:00.000Z,
  RequiredDate: 2026-05-18T00:00:00.000Z
},
Orderdetails: [
  {
    UnitPrice: 18,
    Quantity: 10,
    Discount: 0.05,
    Value: 180,
    product: {
      ProductID: 1,
      ProductName: 'Chai',
      QuantityPerUnit: '10 boxes x 20 bags',
      CategoryID: 1,
      CategoryName: 'Beverages'
    }
  },
  {
    UnitPrice: 31,
    Quantity: 5,
    Discount: 0.05,
    Value: 155,
    product: {
      ProductID: 10,
      ProductName: 'Ikura',
      QuantityPerUnit: '12 - 200 ml jars',
      CategoryID: 8,
      CategoryName: 'Seafood'
    }
  }
],
Freight: 15.5,
'Order Total': 335,
Shipment: {
  Shipper: {
    ShipperID: 1,
    CompanyName: 'Speedy Express'
  },
  ShipName: "Alfred's Futterkiste",
  ShipAddress: 'Obere Str. 57',
  ShipCity: 'Berlin',
  ShipCountry: 'Germany'
}
}
```

Zadanie 2 - modelowanie danych

Zaproponuj strukturę bazy danych dla wybranego/przykładowego zagadnienia/problemu

Należy wybrać jedno zagadnienie/problem (A lub B lub C)

Przykład A

- Wykładowcy, przedmioty, studenci, oceny
 - Wykładowcy prowadzą zajęcia z poszczególnych przedmiotów
 - Studenci uczęszczają na zajęcia
 - Wykładowcy wystawiają oceny studentom
 - Studenci oceniają zajęcia

Przykład B

- Firmy, wycieczki, osoby
 - Firmy organizują wycieczki
 - Osoby rezerwują miejsca/wykupują bilety
 - Osoby oceniają wycieczki

Przykład C

- Własny przykład o podobnym stopniu złożoności

a) Zaproponuj różne warianty struktury bazy danych i dokumentów w poszczególnych kolekcjach oraz przeprowadzić dyskusję każdego wariantu (wskazać wady i zalety każdego z wariantów)

- zdefiniuj schemat/reguły walidacji danych
- wykorzystaj referencje
- dokumenty zagnieżdżone
- tablice

b) Kolekcje należy wypełnić przykładowymi danymi

c) W kontekście zaprezentowania wad/zalet należy zaprezentować kilka przykładów/zapytań/operacji oraz dla których dedykowany jest dany wariant

W sprawozdaniu należy zamieścić przykładowe dokumenty w formacie JSON (pkt a) i b)), oraz kod zapytań/operacji (pkt c)), wraz z odpowiednim komentarzem opisującym strukturę dokumentów oraz polecenia ilustrujące wykonanie przykładowych operacji na danych

Do sprawozdania należy dołączyć

- plik z kodem operacji/zapytań w wersji źródłowej (np. plik .js, np. plik .md)
- oraz kompletny zrzut wykonanych/przygotowanych baz danych (taki zrzut można wykonać np. za pomocą poleceń `js db.orders.aggregate([ // dane klienta { $lookup: { from: "customers", localField: "CustomerID", foreignField: "CustomerID", as: "CustomerData" } }, { $unwind: "$CustomerData" },`

```
// dane pracownika { $lookup: { from: "employees", localField: "EmployeeID", foreignField: "EmployeeID", as: "EmployeeData" } }, { $unwind: "$EmployeeData" },
```

```
// Dołączamy przewoźnika { $lookup: { from: "shippers", localField: "ShipVia", foreignField: "ShipperID", as: "ShipperData" } }, { $unwind: { path: "$ShipperData",
```

```

preserveNullAndEmptyArrays: true } },

// Dołączamy szczegóły zamówienia (orderdetails) { $lookup: { from: "orderdetails", localField:
"OrderID", foreignField: "OrderID", as: "OrderDetailsArray" } }, { $unwind: "$OrderDetailsArray" },

// Dołączamy produkt do zamówienia { $lookup: { from: "products", localField:
"OrderDetailsArray.ProductID", foreignField: "ProductID", as: "ProductData" } }, { $unwind:
"$ProductData" },

// Dołączamy kategorię do produktu { $lookup: { from: "categories", localField:
"ProductData.CategoryID", foreignField: "CategoryID", as: "CategoryData" } }, { $unwind:
"$CategoryData" },

// Obliczamy wartość (Value = UnitPrice * Quantity * (1 - Discount))i budujemy obiekt "product" {
$addFields: { "OrderDetailsArray.Value": { $multiply: [ "$OrderDetailsArray.UnitPrice",
"$OrderDetailsArray.Quantity", { $subtract: [1, "$OrderDetailsArray.Discount"] } ] } },
"OrderDetailsArray.product": { ProductID: "$ProductData.ProductID", ProductName:
"$ProductData.ProductName", QuantityPerUnit: "$ProductData.QuantityPerUnit", CategoryID:
"$CategoryData.CategoryID", CategoryName: "$CategoryData.CategoryName" } } },

// Zwijamy wszystko z powrotem do pojedynczego zamówienia i układamy strukturę { $group: { _id:
"$_id", OrderID: { $first: "$OrderID" }, Customer: { $first: { CustomerID:
"$CustomerData.CustomerID", CompanyName: "$CustomerData.CompanyName", City:
"$CustomerData.City", Country: "$CustomerData.Country" } }, Employee: { $first: { EmployeeID:
"$EmployeeData.EmployeeID", FirstName: "$EmployeeData.FirstName", LastName:
"$EmployeeData.LastName", Title: "$EmployeeData.Title" } }, Dates: { $first: { OrderDate:
"$OrderDate", RequiredDate: "$RequiredDate" } }, Orderdetails: { $push: { UnitPrice:
"$OrderDetailsArray.UnitPrice", Quantity: "$OrderDetailsArray.Quantity", Discount:
"$OrderDetailsArray.Discount", Value: "$OrderDetailsArray.Value", product:
"$OrderDetailsArray.product" } }, Freight: { $first: "$Freight" }, OrderTotal: { $sum:
"$OrderDetailsArray.Value" }, Shipment: { $first: { Shipper: { ShipperID: "$ShipperData.ShipperID",
CompanyName: "$ShipperData.CompanyName" }, ShipName: "$ShipName", ShipAddress:
"$ShipAddress", ShipCity: "$ShipCity", ShipCountry: "$ShipCountry" } } } },

// ostateczny wygląd zgodny z wynikiem { $project: { _id: 1, OrderID: 1, Customer: 1, Employee: 1,
Dates: 1, Orderdetails: 1, Freight: 1, "Order Total": "$OrderTotal", Shipment: 1 } },

// Zapisujemy wynik agregacji do nowej kolekcji OrdersInfo { $out: "OrdersInfo" } })

```

```

Polecenie weryfikujące poprawność
``js
db.OrdersInfo.find().limit(1).pretty()

```

Wyniki:

```

{
  _id: ObjectId('63a060b9bb3b972d6f4e217d'),

```



```
OrderID: 10687,
Customer: {
  CustomerID: 'HUNGO',
  CompanyName: 'Hungry Owl All-Night Grocers',
  City: 'Cork',
  Country: 'Ireland'
},
Employee: {
  EmployeeID: 9,
  FirstName: 'Anne',
  LastName: 'Dodsworth',
  Title: 'Sales Representative'
},
Dates: {
  OrderDate: 1997-09-30T00:00:00.000Z,
  RequiredDate: 1997-10-28T00:00:00.000Z
},
Orderdetails: [
  {
    UnitPrice: 97,
    Quantity: 50,
    Discount: 0.25,
    Value: 3637.5,
    product: {
      ProductID: 9,
      ProductName: 'Mishi Kobe Niku',
      QuantityPerUnit: '18 - 500 g pkgs.',
      CategoryID: 6,
      CategoryName: 'Meat/Poultry'
    }
  },
  {
    UnitPrice: 123.79,
    Quantity: 10,
    Discount: 0,
    Value: 1237.9,
    product: {
      ProductID: 29,
      ProductName: 'Thüringer Rostbratwurst',
      QuantityPerUnit: '50 bags x 30 sausgs.',
      CategoryID: 6,
      CategoryName: 'Meat/Poultry'
    }
  },
  {
    UnitPrice: 19,
    Quantity: 6,
    Discount: 0.25,
    Value: 85.5,
    product: {
      ProductID: 36,
      ProductName: 'Inlagd Sill',
      QuantityPerUnit: '24 - 250 g jars',
      CategoryID: 8,
```

```

        CategoryName: 'Seafood'
      }
    }
  ],
  Freight: 296.43,
  Shipment: {
    Shipper: {
      ShipperID: 2,
      CompanyName: 'United Package'
    },
    ShipName: 'Hungry Owl All-Night Grocers',
    ShipAddress: '8 Johnstown Road',
    ShipCity: 'Cork',
    ShipCountry: 'Ireland'
  },
  'Order Total': 4960.9
}

```

b)

```

db.customers.aggregate([
  // Dołączamy całą listę zamówień z wcześniej stworzonej kolekcji
  OrdersInfo
  {
    $lookup: {
      from: "OrdersInfo",
      localField: "CustomerID",
      foreignField: "Customer.CustomerID",
      as: "Orders"
    }
  },

  // ostateczny wygląd dokumentu i usuwamy zduplikowane dane klienta z
  zamówień
  {
    $project: {
      _id: 1,
      CustomerID: 1,
      CompanyName: 1,
      City: 1,
      Country: 1,
      "Orders._id": 1,
      "Orders.OrderID": 1,
      "Orders.Employee": 1,
      "Orders.Dates": 1,
      "Orders.Orderdetails": 1,
      "Orders.Freight": 1,
      "Orders.Order Total": 1,
      "Orders.Shipment": 1
    }
  },

```

```
// Zapisujemy wynik do kolekcji CustomerInfo
{ $out: "CustomerInfo" }
])
```

Polecenie weryfikujące poprawność

```
db.CustomerInfo.find().limit(1).pretty()
```

Wyniki:

```
{
  _id: ObjectId('63a05cdfbb3b972d6f4e097b'),
  CustomerID: 'ALFKI',
  CompanyName: 'Alfreds Futterkiste',
  City: 'Berlin',
  Country: 'Germany',
  Orders: [ // tablica zamówień klienta o strukturze takiej jak w punkcie
a) (oczywiście bez informacji o kliencie) (dodałem tutaj jedno zamówienie
dla czytelności)
    {
      _id: ObjectId('63a060b9bb3b972d6f4e218c'),
      OrderID: 10702,
      Employee: {
        EmployeeID: 4,
        FirstName: 'Margaret',
        LastName: 'Peacock',
        Title: 'Sales Representative'
      },
      Dates: {
        OrderDate: 1997-10-13T00:00:00.000Z,
        RequiredDate: 1997-11-24T00:00:00.000Z
      },
      Orderdetails: [
        {
          UnitPrice: 10,
          Quantity: 6,
          Discount: 0,
          Value: 60,
          product: {
            ProductID: 3,
            ProductName: 'Aniseed Syrup',
            QuantityPerUnit: '12 - 550 ml bottles',
            CategoryID: 2,
            CategoryName: 'Condiments'
          }
        },
        {
          UnitPrice: 18,
          Quantity: 15,
```

```

        Discount: 0,
        Value: 270,
        product: {
            ProductID: 76,
            ProductName: 'Lakkalikööri',
            QuantityPerUnit: '500 ml',
            CategoryID: 1,
            CategoryName: 'Beverages'
        }
    },
    Freight: 23.94,
    Shipment: {
        Shipper: {
            ShipperID: 1,
            CompanyName: 'Speedy Express'
        },
        ShipName: "Alfred's Futterkiste",
        ShipAddress: 'Obere Str. 57',
        ShipCity: 'Berlin',
        ShipCountry: 'Germany'
    },
    'Order Total': 330
},
]
}

```

c)

Użycie oryginalnych kolekcji wymagałoby bardzo złożonego zapytania z kilkoma złączeniami (\$lookup), co jest powolne. Z kolekcją OrdersInfo byłoby łatwiej, ale nadal wymagałoby dodatkowego grupowania. Wybrane przeze mnie rozwiązanie na kolekcji CustomerInfo jest najprostsze i najszybsze – wystarczy tylko rozwinąć zagnieżdżone tablice, przefiltrować dane i zsumować wyniki.

```

db.CustomerInfo.aggregate([
    // Rozwijamy tablicę zamówień klienta
    { $unwind: "$Orders" },

    // Filtrujemy tylko zamówienia z 1997 roku
    {
        $match: {
            "Orders.Dates.OrderDate": {
                $gte: ISODate("1997-01-01T00:00:00Z"),
                $lt: ISODate("1998-01-01T00:00:00Z")
            }
        }
    },

    // Rozwijamy pozycje w zamówieniu, aby sprawdzić kategorie produktów
    { $unwind: "$Orders.Orderdetails" },

```

```
// Filtrujemy tylko produkty z kategorii Confections
{
  $match: {
    "Orders.Orderdetails.product.CategoryName": "Confections"
  }
},

// Grupujemy i sumujemy wartość
{
  $group: {
    _id: "$_id",
    CustomerID: { $first: "$CustomerID" },
    CompanyName: { $first: "$CompanyName" },
    ConfectionsSale97: { $sum: "$Orders.Orderdetails.Value" }
  }
},

// Formatujemy wynik
{
  $project: {
    _id: 1,
    CustomerID: 1,
    CompanyName: 1,
    ConfectionsSale97: 1
  }
}
])
```

Przykładowe wyniki:

```
{
  _id: ObjectId('63a05cdfbb3b972d6f4e097e'),
  CustomerID: 'AROUT',
  CompanyName: 'Around the Horn',
  ConfectionsSale97: 375.19999977201223
}
{
  _id: ObjectId('63a05cdfbb3b972d6f4e09c1'),
  CustomerID: 'SAVEA',
  CompanyName: 'Save-a-lot Markets',
  ConfectionsSale97: 6351.084993118047
}
```

d)

Podobnie jak wcześniej, najwydajniejsza jest gotowa kolekcja CustomerInfo, ponieważ pozwala nam uniknąć czasochłonnego łączenia tabel. Żeby uzyskać podział na lata i miesiące, po prostu wyciągamy daty z zamówień i wykonujemy dwa szybkie grupowania (\$group): najpierw sumując sprzedaż dla danego miesiąca, a potem zwijając to wszystko do jednej tablicy dla każdego klienta.

```
db.CustomerInfo.aggregate([
  // Rozwijamy zamówienia
  { $unwind: "$Orders" },

  // Rozwijamy szczegóły zamówień, aby uzyskać wartość
  { $unwind: "$Orders.Orderdetails" },

  // Sumujemy sprzedaż dla każdej kombinacji (Klient + Rok + Miesiąc)
  {
    $group: {
      _id: {
        CustomerID: "$CustomerID",
        CompanyName: "$CompanyName",
        Year: { $year: { $toDate: "$Orders.Dates.OrderDate" } },
        Month: { $month: { $toDate: "$Orders.Dates.OrderDate" } }
      },
      MonthlyTotal: { $sum: "$Orders.Orderdetails.Value" }
    }
  },

  // Zwijamy wyliczone miesiące z powrotem do tablicy 'Sale' dla klienta
  {
    $group: {
      _id: "$_id.CustomerID",
      CustomerID: { $first: "$_id.CustomerID" },
      CompanyName: { $first: "$_id.CompanyName" },
      Sale: {
        $push: {
          Year: "$_id.Year",
          Month: "$_id.Month",
          Total: "$MonthlyTotal"
        }
      }
    }
  },

  // Formatowanie wyglądu
  {
    $project: {
      _id: 0,
      CustomerID: 1,
      CompanyName: 1,
      Sale: 1
    }
  }
])
```

Przykładowe wyniki:

```
{
  CustomerID: 'LILAS',
  CompanyName: 'LILA-Supermercado',
  Sale: [
    {
      Year: 1998,
      Month: 4,
      Total: 1885
    },
    {
      Year: 1997,
      Month: 4,
      Total: 1412
    },
    {
      Year: 1996,
      Month: 10,
      Total: 1648.999988436699
    },
    {
      Year: 1998,
      Month: 5,
      Total: 673.9199996200203
    },
    {
      Year: 1997,
      Month: 12,
      Total: 720
    },
    {
      Year: 1996,
      Month: 9,
      Total: 1050.6
    },
    {
      Year: 1996,
      Month: 12,
      Total: 112
    },
    {
      Year: 1996,
      Month: 11,
      Total: 1167.6799971342086
    },
    {
      Year: 1997,
      Month: 5,
      Total: 1504.4999894499779
    },
    {
      Year: 1996,
      Month: 8,
```

```

    Total: 1414.8
  },
  {
    Year: 1998,
    Month: 1,
    Total: 2825.999995805323
  },
  {
    Year: 1998,
    Month: 2,
    Total: 122.39999914169312
  },
  {
    Year: 1997,
    Month: 2,
    Total: 1538.7
  }
]
}

```

e)

- Aktualizacja oryginalnych kolekcji (orders, orderdetails):

```

// Dodanie nagłówka zamówienia
db.orders.insertOne({
  OrderID: 99999,
  CustomerID: "ALFKI",
  EmployeeID: 1,
  OrderDate: new ISODate("2026-05-04T00:00:00Z"),
  ShipVia: 1,
  Freight: 15.50,
  ShipName: "Alfred's Futterkiste",
  ShipAddress: "Obere Str. 57",
  ShipCity: "Berlin",
  ShipCountry: "Germany"
});

// Dodanie dwóch produktów do zamówienia
db.orderdetails.insertMany([
  { OrderID: 99999, ProductID: 1, UnitPrice: 18.00, Quantity: 10, Discount: 0 },
  { OrderID: 99999, ProductID: 10, UnitPrice: 31.00, Quantity: 5, Discount: 0 }
]);

```

- Aktualizacja kolekcji OrdersInfo:

```

// Wstawienie całego dokumentu
db.OrdersInfo.insertOne({

```



```
OrderID: 99999,
Customer: {
  CustomerID: "ALFKI",
  CompanyName: "Alfreds Futterkiste",
  City: "Berlin",
  Country: "Germany"
},
Employee: {
  EmployeeID: 1,
  FirstName: "Nancy",
  LastName: "Davolio",
  Title: "Sales Representative"
},
Dates: {
  OrderDate: new ISODate("2026-05-04T00:00:00Z"),
  RequiredDate: new ISODate("2026-05-18T00:00:00Z")
},
Orderdetails: [
  {
    UnitPrice: 18.00,
    Quantity: 10,
    Discount: 0,
    Value: 180.00,
    product: {
      ProductID: 1,
      ProductName: "Chai",
      QuantityPerUnit: "10 boxes x 20 bags",
      CategoryID: 1,
      CategoryName: "Beverages"
    }
  },
  {
    UnitPrice: 31.00,
    Quantity: 5,
    Discount: 0,
    Value: 155.00,
    product: {
      ProductID: 10,
      ProductName: "Ikura",
      QuantityPerUnit: "12 - 200 ml jars",
      CategoryID: 8,
      CategoryName: "Seafood"
    }
  }
],
Freight: 15.50,
"Order Total": 335.00,
Shipment: {
  Shipper: {
    ShipperID: 1,
    CompanyName: "Speedy Express"
  },
  ShipName: "Alfred's Futterkiste",
  ShipAddress: "Obere Str. 57",
```

```
    ShipCity: "Berlin",
    ShipCountry: "Germany"
  }
});
```

- Aktualizacja kolekcji CustomerInfo:

```
// Wypchnięcie nowego zamówienia do istniejącej tablicy Orders klienta ALFKI
db.CustomerInfo.updateOne(
  { CustomerID: "ALFKI" },
  {
    $push: {
      Orders: {
        OrderID: 99999,
        Employee: {
          EmployeeID: 1,
          FirstName: "Nancy",
          LastName: "Davolio",
          Title: "Sales Representative"
        },
        Dates: {
          OrderDate: new ISODate("2026-05-04T00:00:00Z"),
          RequiredDate: new ISODate("2026-05-18T00:00:00Z")
        },
        Orderdetails: [
          {
            UnitPrice: 18.00,
            Quantity: 10,
            Discount: 0,
            Value: 180.00,
            product: {
              ProductID: 1,
              ProductName: "Chai",
              QuantityPerUnit: "10 boxes x 20 bags",
              CategoryID: 1,
              CategoryName: "Beverages"
            }
          },
          {
            UnitPrice: 31.00,
            Quantity: 5,
            Discount: 0,
            Value: 155.00,
            product: {
              ProductID: 10,
              ProductName: "Ikura",
              QuantityPerUnit: "12 - 200 ml jars",
              CategoryID: 8,
              CategoryName: "Seafood"
            }
          }
        ]
      }
    }
  }
});
```

```

    ],
    Freight: 15.50,
    "Order Total": 335.00,
    Shipment: {
      Shipper: {
        ShipperID: 1,
        CompanyName: "Speedy Express"
      },
      ShipName: "Alfred's Futterkiste",
      ShipAddress: "Obere Str. 57",
      ShipCity: "Berlin",
      ShipCountry: "Germany"
    }
  }
}
);

```

Polecenie weryfikujące poprawność

```

db.CustomerInfo.find(
  { CustomerID: "ALFKI" },
  { Orders: { $slice: -1 } } // ostatni element
).pretty()

```

Wyniki:

```

{
  _id: ObjectId('63a05cdfbb3b972d6f4e097b'),
  CustomerID: 'ALFKI',
  CompanyName: 'Alfreds Futterkiste',
  City: 'Berlin',
  Country: 'Germany',
  Orders: [
    {
      OrderID: 99999,
      Employee: {
        EmployeeID: 1,
        FirstName: 'Nancy',
        LastName: 'Davolio',
        Title: 'Sales Representative'
      },
      Dates: {
        OrderDate: 2026-05-04T00:00:00.000Z,
        RequiredDate: 2026-05-18T00:00:00.000Z
      },
      Orderdetails: [
        {
          UnitPrice: 18,

```

```

    Quantity: 10,
    Discount: 0,
    Value: 180,
    product: {
      ProductID: 1,
      ProductName: 'Chai',
      QuantityPerUnit: '10 boxes x 20 bags',
      CategoryID: 1,
      CategoryName: 'Beverages'
    }
  },
  {
    UnitPrice: 31,
    Quantity: 5,
    Discount: 0,
    Value: 155,
    product: {
      ProductID: 10,
      ProductName: 'Ikura',
      QuantityPerUnit: '12 - 200 ml jars',
      CategoryID: 8,
      CategoryName: 'Seafood'
    }
  }
],
Freight: 15.5,
'Order Total': 335,
Shipment: {
  Shipper: {
    ShipperID: 1,
    CompanyName: 'Speedy Express'
  },
  ShipName: "Alfred's Futterkiste",
  ShipAddress: 'Obere Str. 57',
  ShipCity: 'Berlin',
  ShipCountry: 'Germany'
}
}
]
}

```

f)

- Aktualizacja w oryginalnej kolekcji **orderdetails**:

```

db.orderdetails.updateMany(
  { OrderID: 99999 },
  { $inc: { Discount: 0.05 } }
);

```

- Aktualizacja w kolekcji **OrdersInfo**:

```
db.OrdersInfo.updateOne(
  { OrderID: 99999 },
  { $inc: { "Orderdetails.$[].Discount": 0.05 } }
  // $[] sprawia, że zmiana dotyczy każdego elementu w tablicy Orderdetails
);
```

- Aktualizacja w kolekcji **CustomerInfo**:

```
db.CustomerInfo.updateOne(
  { CustomerID: "ALFKI" },
  { $inc: { "Orders.$[order].Orderdetails.$[].Discount": 0.05 } },
  { arrayFilters: [ { "order.OrderID": 99999 } ] }
  // arrayFilters wybiera zamówienie o ID 99999 z tablicy Orders, a
  następnie modyfikujemy wszystkie ($[]) pozycje w jego Orderdetails
);
```

Polecenie weryfikujące poprawność

```
db.CustomerInfo.find(
  { CustomerID: "ALFKI" },
  { Orders: { $slice: -1 } } //ostatni element
).pretty()
```

Wyniki:

```
{
  _id: ObjectId('63a05cdfbb3b972d6f4e097b'),
  CustomerID: 'ALFKI',
  CompanyName: 'Alfreds Futterkiste',
  City: 'Berlin',
  Country: 'Germany',
  Orders: [
    {
      OrderID: 99999,
      Employee: {
        EmployeeID: 1,
        FirstName: 'Nancy',
        LastName: 'Davolio',
        Title: 'Sales Representative'
      },
      Dates: {
        OrderDate: 2026-05-04T00:00:00.000Z,
        RequiredDate: 2026-05-18T00:00:00.000Z
      },
      Orderdetails: [
        {
```

```
    UnitPrice: 18,  
    Quantity: 10,  
    Discount: 0.05,  
    Value: 180,  
    product: {  
      ProductID: 1,  
      ProductName: 'Chai',  
      QuantityPerUnit: '10 boxes x 20 bags',  
      CategoryID: 1,  
      CategoryName: 'Beverages'  
    }  
  },  
  {  
    UnitPrice: 31,  
    Quantity: 5,  
    Discount: 0.05,  
    Value: 155,  
    product: {  
      ProductID: 10,  
      ProductName: 'Ikura',  
      QuantityPerUnit: '12 - 200 ml jars',  
      CategoryID: 8,  
      CategoryName: 'Seafood'  
    }  
  }  
],  
Freight: 15.5,  
'Order Total': 335,  
Shipment: {  
  Shipper: {  
    ShipperID: 1,  
    CompanyName: 'Speedy Express'  
  },  
  ShipName: "Alfred's Futterkiste",  
  ShipAddress: 'Obere Str. 57',  
  ShipCity: 'Berlin',  
  ShipCountry: 'Germany'  
}  
}  
]
```

Zadanie 2 - modelowanie danych

Zaproponuj strukturę bazy danych dla wybranego/przykładowego zagadnienia/problemu

Należy wybrać jedno zagadnienie/problem (A lub B lub C)

Przykład A

- Wykładowcy, przedmioty, studenci, oceny

- Wykładowcy prowadzą zajęcia z poszczególnych przedmiotów
- Studenci uczęszczają na zajęcia
- Wykładowcy wystawiają oceny studentom
- Studenci oceniają zajęcia

Przykład B

- Firmy, wycieczki, osoby
 - Firmy organizują wycieczki
 - Osoby rezerwują miejsca/wykupują bilety
 - Osoby oceniają wycieczki

Przykład C

- Własny przykład o podobnym stopniu złożoności

a) Zaproponuj różne warianty struktury bazy danych i dokumentów w poszczególnych kolekcjach oraz przeprowadź dyskusję każdego wariantu (wskazać wady i zalety każdego z wariantów)

- zdefiniuj schemat/reguły walidacji danych
- wykorzystaj referencje
- dokumenty zagnieżdżone
- tablice

b) Kolekcje należy wypełnić przykładowymi danymi

c) W kontekście zaprezentowania wad/zalet należy zaprezentować kilka przykładów/zapytań/operacji oraz dla których dedykowany jest dany wariant

W sprawozdaniu należy zamieścić przykładowe dokumenty w formacie JSON (pkt a) i b)), oraz kod zapytań/operacji (pkt c)), wraz z odpowiednim komentarzem opisującym strukturę dokumentów oraz polecenia ilustrujące wykonanie przykładowych operacji na danych

Do sprawozdania należy dołączyć

- plik z kodem operacji/zapytań w wersji źródłowej (np. plik .js, np. plik .md)
- oraz kompletny zrzut wykonanych/przygotowanych baz danych (taki zrzut można wykonać np. za pomocą poleceń `mongoexport`, `mongodump` ...)
 - załącznik ze zrzutem baz powinien mieć format zip

Zadanie 2 - rozwiązanie

Wybraliśmy przykład B do implementacji

a)

W przypadku modelowania bazy dla wycieczek organizowanych przez firmy oraz rezerwowanych i ocenianych przez osoby, możemy rozważyć dwa podejścia:

Wariant 1: Model silnie znormalizowany (Relacyjny)

Struktura: Tworzymy 5 oddzielnych kolekcji: companies (firmy), people (osoby), trips (wycieczki), bookings (rezerwacje), reviews (oceny). Łączymy je wyłącznie za pomocą referencji (np. w kolekcji reviews trzymamy tylko trip_id i person_id).

Zalety: Całkowity brak duplikacji danych. Jeśli firma zmieni nazwę, modyfikujemy to tylko w jednym dokumencie w kolekcji companies.

Wady: Aby pobrać informacje o wycieczce (np. jej nazwę, nazwę firmy organizującej oraz wszystkie oceny), musimy użyć wielu drogich operacji złączeń (\$lookup). W MongoDB jest to mało wydajne i nie wykorzystuje pełnego potencjału bazy dokumentowej.

Wariant 2: Model hybrydowy (Dokumentowy)

Struktura: Tworzymy tylko 3 kolekcje: companies, people oraz centralną kolekcję trips. W kolekcji trips przechowujemy podstawowe referencje, ale też stosujemy tablice i dokumenty zagnieżdżone.

Wykorzystanie mechanizmów:

Dokumenty zagnieżdżone (embedded documents): Podstawowe dane organizatora (id oraz nazwa firmy) są zagnieżdżone bezpośrednio w dokumencie wycieczki.

Tablice (arrays): Oceny (reviews) to tablica zagnieżdżonych obiektów wewnątrz wycieczki.

Referencje: Rezerwacje to tablica referencji (identyfikatorów) do klientów z kolekcji people.

Zalety: Błyskawiczny odczyt. Większość kluczowych informacji o wycieczce pobieramy pojedynczym zapytaniem (bez \$lookup), co idealnie pasuje do architektury aplikacji wyświetlającej oferty.

Wady: Denormalizacja – jeśli firma zmieni nazwę, trzeba ją zaktualizować nie tylko w companies, ale też we wszystkich zagnieżdżonych dokumentach w trips.

Schemat i reguły walidacji danych (dla kolekcji trips)

Przed dodaniem danych, definiujemy reguły walidacji (\$jsonSchema) dla kolekcji docelowej.

```
db.createCollection("trips", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [ "title", "price", "organizer" ],
      properties: {
        title: {
          bsonType: "string",
          description: "Tytuł wycieczki – wymagany ciąg znaków"
        },
        price: {
          bsonType: "number",
          minimum: 0,
          description: "Cena – wymagana liczba dodatnia"
        },
        organizer: {
          bsonType: "object",
```



```

        required: [ "company_id", "company_name" ],
        description: "Dokument zagnieżdżony – dane firmy
organizującej"
    },
    booked_by: {
        bsonType: "array",
        items: { bsonType: "int" },
        description: "Tablica referencji – lista ID osób, które
wykupiły bilety"
    },
    reviews: {
        bsonType: "array",
        description: "Tablica zagnieżdżonych dokumentów – oceny
wycieczki"
    }
}
}
}
});

```

b)

Wypełniamy kolekcje słownikowe (companies, people) oraz główną kolekcję (trips) z wykorzystaniem wymogów strukturalnych (referencje, zagnieżdżenia, tablice).

```

// 1. Dodawanie firm organizujących wycieczki
db.companies.insertMany([
  { _id: 1, name: "Góry-Travel", contact: "kontakt@gory-travel.pl" },
  { _id: 2, name: "Morze-Adventures", contact: "biuro@morze.pl" }
]);

// 2. Dodawanie osób (klientów)
db.people.insertMany([
  { _id: 101, firstname: "Jan", lastname: "Kowalski", email:
"jan@example.com" },
  { _id: 102, firstname: "Anna", lastname: "Nowak", email:
"anna@example.com" },
  { _id: 103, firstname: "Piotr", lastname: "Zalewski", email:
"piotr@example.com" }
]);

// 3. Dodawanie wycieczek
db.trips.insertMany([
  {
    _id: 1001,
    title: "Weekend w Tatrach",
    price: 450,
    organizer: {
      company_id: 1,
      company_name: "Góry-Travel"
    }
  },

```

```

    booked_by: [101, 102],
    reviews: [
      { person_id: 101, rating: 5, comment: "Świetna wycieczka, polecam!" },
      { person_id: 102, rating: 4, comment: "Pogoda nie dopisała, ale organizacja super." }
    ]
  },
  {
    _id: 1002,
    title: "Rejs po Bałtyku",
    price: 1200,
    organizer: {
      company_id: 2,
      company_name: "Morze-Adventures"
    },
    booked_by: [103],
    reviews: []
  }
]);

```

c)

Przykład 1: Bardzo szybki odczyt strony wycieczki (ZALETA) Ten typ zapytania jest bardzo wydajny, ponieważ dzięki dokumentom zagnieżdżonym i tablicom pobieramy wszystkie szczegóły oferty, organizatora i opinie bez użycia złożonych operacji łączenia relacji.

```

db.trips.aggregate([
  { $match: { _id: 1001 } },
  { $project: {
    _id: 0,
    title: 1,
    organizerName: "$organizer.company_name",
    totalBookings: { $size: "$booked_by" },
    averageRating: { $avg: "$reviews.rating" }
  }
}
]);

```

Przykład 2: Pobranie pełnych danych osób, które zarezerwowały wycieczkę (WADA/TRUDNOŚĆ)

Ponieważ przechowujemy tylko referencje (id uczestników), aby pobrać ich pełne nazwiska, musimy użyć operatora \$lookup, co pokazuje, że pewne elementy relacyjne nadal istnieją w modelu dokumentowym.

```

db.trips.aggregate([
  { $match: { _id: 1001 } },
  { $lookup: {

```

```

        from: "people",
        localField: "booked_by",
        foreignField: "_id",
        as: "participants_details"
    }
},
{ $project: {
    title: 1,
    "participants_details.firstname": 1,
    "participants_details.lastname": 1
}
}
});

```

Przykład 3: Dodanie nowej rezerwacji miejsca (ZALETA) Dodanie klienta do wycieczki sprowadza się do prostej operacji na tablicy (\$push).

```

db.trips.updateOne(
{ _id: 1002 },
{ $push: { booked_by: 101 } }
);

```

TESTY

Test 1:

```

db.trips.aggregate([
{ $match: { _id: 1001 } },
{ $project: { _id: 0, title: 1, organizerName: "$organizer.company_name",
totalBookings: { $size: "$booked_by" }, averageRating: { $avg:
"$reviews.rating" } } }
]);

```

```

>_MONGOSH
> db.trips.aggregate([
  { $match: { _id: 1001 } },
  { $project: { _id: 0, title: 1, organizerName: "$organizer.company_name", totalBookings: { $size: "$booked_by" }, averageRating: {
  ]});
< {
  title: 'Weekend w Tatrach',
  organizerName: 'Góry-Travel',
  totalBookings: 2,
  averageRating: 4.5
}

```

Test 2:

```
db.trips.aggregate([
  { $match: { _id: 1001 } },
  { $lookup: { from: "people", localField: "booked_by", foreignField:
    "_id", as: "participants_details" } },
  { $project: { title: 1, "participants_details.firstname": 1,
    "participants_details.lastname": 1 } }
]);
```

```
> db.trips.aggregate([
  { $match: { _id: 1001 } },
  { $lookup: { from: "people", localField: "booked_by", foreignField: "_id", as: "participants_details" } },
  { $project: { title: 1, "participants_details.firstname": 1, "participants_details.lastname": 1 } }
]);
< {
  _id: 1001,
  title: 'Weekend w Tatrach',
  participants_details: [
    {
      firstname: 'Jan',
      lastname: 'Kowalski'
    },
    {
      firstname: 'Anna',
      lastname: 'Nowak'
    }
  ]
}
```

Test 3:

```
db.trips.updateOne(
  { _id: 1002 },
  { $push: { booked_by: 101 } }
);
```

```
> db.trips.updateOne(
  { _id: 1002 },
  { $push: { booked_by: 101 } }
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```